COSC726

AI EDUCATIONAL TUTOR

Khanmigo-Inspired Agentic Educational Tutor

Project information	Details
Module	COSC726
Project title	AI Educational Tutor
Student name	Ayat abdo alhay fadoal
GitHub repository URL	[ADD PUBLIC REPOSITORY URL]
YouTube demonstration URL	[ADD UNLISTED VIDEO URL]

Abstract

This report presents the design, implementation, and component-level evaluation of AI Educational Tutor, a Khanmigo-inspired agentic educational tutoring application. The system is intended to support learning through guided explanations, practice activities, quizzes, and the use of computational and knowledge-retrieval tools. Rather than treating the application as a general-purpose question-answering chatbot, the project organizes the tutoring process as a bounded workflow: the learner submits a request, the system interprets the learning intent, selects an action, executes an available tool or generates a pedagogical response, observes the result, reflects on whether further work is needed, and validates the output before presenting it.

The implementation uses Python, Streamlit, LangGraph, SQLite, Pydantic, and a locally hosted Qwen2.5 3B model served through Ollama. The application covers Mathematics, Algorithms, Data Structures, and broader Computer Science learning topics. A component-level test suite contains nine tests for input validation and student-memory behaviour. The accompanying evaluation runner executes nine corresponding tasks and, in the recorded run, reported nine passes, zero failures, and a 100.00% pass rate. These results demonstrate that the tested components behaved as expected in that run; they do not establish end-to-end tutoring accuracy, educational effectiveness, or reliability of every agent workflow. The report documents the architecture, implementation choices, evidence, responsible-design considerations, limitations, and future work.

1. Problem and Domain Fit

1.1 Educational problem

Students studying technical subjects often need more than a final answer. They may need a concept explained in plain language, a worked example, a hint that helps them make the next step, or a practice question that reveals whether they understand the underlying idea. A conventional chatbot can respond to questions, but a tutoring system also needs to consider the learner's current topic, the type of help requested, the state of an ongoing quiz or exercise, and whether the response is appropriate for learning.

The project addresses this need by providing an interactive tutor for selected areas of Mathematics and Computer Science. Examples include Linear Equations, Linear Search, Binary Search, Arrays, Sorting, and related algorithmic concepts. The application is designed to encourage the learner to participate in the reasoning process. In a tutoring context, immediately revealing an answer may be less useful than offering a hint, asking a guiding question, or presenting a practice task. The system therefore includes distinct interaction paths for explanations, hints, practice, and quizzes.

1.2 Why an agentic approach is appropriate

An agentic workflow is appropriate because tutoring requests are not all the same. A request to calculate an expression may require a calculator; a request to explain Linear Search may benefit from retrieved learning material and a pedagogical explanation; a quiz request requires question generation and answer verification; and a submitted answer must be interpreted in the context of a pending activity. A single unstructured language model response would not reliably distinguish and coordinate these different tasks.

The application therefore uses an agentic controller to route requests through a sequence of specialised stages. The controller provides structure around the language model and tools. It also imposes iteration limits and validation gates. This is a bounded form of agentic behaviour: the system can select among available actions and react to their results, but it does not operate autonomously outside the application or take consequential external actions.

1.3 Intended users and scope

The intended users are students seeking supplementary explanations and practice in introductory Mathematics and Computer Science. The interface supports English and Arabic localisation, although the completeness of translation varies across interface and lesson content. The project is a learning-support prototype, not a replacement for a teacher, formal assessment, or a complete learning-management system. It does not claim to diagnose learning difficulties or to measure mastery with validated educational instruments.

1.4 Project objectives

- Provide a conversational interface for educational questions and learning activities.
- Distinguish common learning intents such as explanation, calculation, exercise generation, quiz generation, and quiz answering.
- Use a graph-based controller to coordinate interpretation, planning, action, observation, reflection, and response validation.
- Connect the tutor to a local language model and selected tools.
- Maintain conversation records and pending quiz or exercise state using SQLite.
- Provide basic student-memory structures and topic-oriented knowledge retrieval.
- Supply reproducible component tests and an evaluation task set that can run without calling the language model.

2. System Design

2.1 High-level architecture

The application is divided into a user interface, agent and graph orchestration, language-model access, knowledge retrieval, memory and persistence, schemas and validation, and tools. The Streamlit interface collects the learner's message and selected subject or topic. The application converts the request into a structured TutorInput object and passes it to the tutor execution function. The LangGraph workflow coordinates the internal stages. Where required, an action is delegated to a tool or to the local language model. The result is then observed, reflected upon, formatted, and passed through output validation.

Layer	Responsibility	Main implementation
User interface	Collect messages, language, subject/topic selections, and render tutoring interactions.	Streamlit; ui/app.py and ui/components/
Agent logic	Interpret requests, plan actions, apply pedagogical response behaviour, and reflect.	agent/
Workflow controller	Execute nodes and conditional routes with bounded iterations.	graph/; LangGraph
LLM integration	Generate and interpret natural-language content using a local model.	llm/model.py; Ollama; Qwen2.5 3B
Knowledge	Load topic data and retrieve relevant material.	knowledge/loader.py; knowledge/retriever.py
Memory and persistence	Maintain learner state, conversation records, tool history,	memory/; SQLite

	and pending activities.	
Tools and schemas	Perform calculations, manage quizzes, and validate structured data.	tools/; schemas/

2.2 Agent workflow

The LangGraph workflow is represented as a StateGraph. Its main sequence is input_gate, understand, plan, action_gate, act, observe, reflect, respond, validate, and output_gate, with a repair route available when validation reports a problem. Conditional routing functions determine whether the request can proceed, which action is permitted, whether the agent should respond or replan, and whether the output can be returned.

The input gate checks basic request properties such as the presence of a student identifier, a non-empty message, the maximum message length, and an acceptable subject. The understanding stage classifies the learner's intent. The planner then selects an action based mainly on rule-based conditions and the interpreted intent. The action gate checks the selected action against an allowlist. The act stage invokes the relevant tool or response path. The observe stage records or interprets the action result. The reflection stage decides whether the available result is sufficient or whether replanning is needed. The response stage produces learner-facing content, while the validation and output gates check that a non-empty response can be returned.

The workflow is bounded by a small maximum iteration count in the tutor execution path. This is important because an agent that can repeatedly replan without a limit may consume excessive time or fail to terminate. The graph also uses a LangGraph recursion limit at execution time. These controls constrain execution, although they do not guarantee that every selected action is pedagogically or factually correct.

2.3 Planning and action selection

The understanding component uses a local language-model classification prompt for several request categories, including conceptual explanation, calculation, code analysis, algorithm analysis, definition, comparison, exercise, quiz, quiz answer, and general learning. Some activity-related conditions are also recognised through phrase-based rules in the graph nodes. The planner itself is primarily rule-based: it maps the interpreted intent and available context to actions such as retrieve_knowledge, calculate, generate_exercise, manage_quiz, analyze_code, or analyze_complexity.

This separation has a practical advantage. The language model can help interpret natural language, while the planner constrains the system to a known set of operations. However, rule-based planning can be brittle when a learner phrases a request in an unexpected way. The supported action list is also broader than the set of actions fully implemented in the reviewed execution path. In particular, run_python and update_progress appear in the action allowlist but were not confirmed as implemented actions in the reviewed act-node logic. They should not be presented as completed capabilities.

2.4 Memory and context

The system uses several forms of state. Conversation context can include recent messages, the selected language, subject and topic, student-memory data, and relevant retrieved knowledge. The context manager organises these values for use by the tutor. The conversation store uses SQLite to persist messages, tool executions, pending quizzes, pending exercises, and exercise history. This persistence supports continuity for activity workflows.

The StudentMemory structure tracks lists of mastered and weak concepts and includes attempts and correct-attempt counters. The tests confirm its initial state and that concepts can be added without duplicate entries. The reviewed update method does not itself increment the attempt counters, calculate mastery from evidence, or

persist student memory to the database. Consequently, these fields should be understood as a basic memory data structure rather than a complete learner model.

2.5 Knowledge retrieval

Knowledge files are loaded from JSON data under the knowledge directory. The retriever uses lexical matching and simple scoring based on subject, topic, concept, definition, explanation, and keyword matches. It is not an embedding-based or vector-database retrieval system. This design is lightweight and understandable for a prototype, but its coverage depends on the quality and completeness of the JSON content and on how closely a learner's wording matches the stored terms.

2.6 Tools and schemas

The calculator evaluates a restricted set of arithmetic expressions by parsing an abstract syntax tree and accepting selected numeric and arithmetic nodes. It does not rely on unrestricted eval execution. Quiz management generates questions through the local model and uses validation logic to check selected question types and expected answers. Pydantic schemas define structured inputs, plans, reflections, responses, and validation outputs. These schemas reduce ambiguity at component boundaries and allow invalid input to be rejected before it enters the workflow.

3. Implementation

3.1 Technology stack and execution environment

Technology	Role in the project
Python 3.12	Primary implementation language; the recorded runtime was Python 3.12.10.
Streamlit	Web-based user interface and interactive controls.
LangGraph	Stateful graph orchestration and conditional workflow routing.
Ollama	Local model-serving runtime.
Qwen2.5 3B	Local language model configured for generation and request interpretation.
Requests	HTTP communication with the local Ollama API.
Pydantic	Input and output schema validation.
SQLite	Local persistence for conversations and pending activities.
pytest	Automated component testing.

The local model integration is implemented in llm/model.py. LocalLLM sends an HTTP POST request to the Ollama generate endpoint at http://localhost:11434/api/generate, with the configured model name qwen2.5:3b. The request uses non-streaming generation and a configured timeout. This arrangement avoids requiring a paid hosted LLM API for the current implementation, but it requires Ollama and the model to be installed and running on the machine that hosts the application.

3.2 Application startup

The documented application command is: python -m streamlit run ui/app.py. The Streamlit application presents the AI Educational Tutor interface and allows a learner to select language, subject, and topic, then interact through the tutor area and action controls. The application includes Mathematics, Algorithms, and Data Structures topic selections, with broader Computer Science support in the tutor input defaults and knowledge content.

3.3 User interaction and learning activities

The interface includes controls for hint, explanation, practice, and quiz interactions. These controls send activity-oriented prompts into the tutor workflow. The quiz path can create a pending question and retain its expected answer so that the learner's next message can be checked against the pending activity. The implementation also limits quiz-generation attempts and applies topic-specific answer validation for supported cases. A recorded manual example used the equation 2x + 3 = 11, for which the expected answer was x = 4; submitting 4 produced a correct-answer response. This is a concrete functional example, not a statistically meaningful measure of quiz quality.

The exercise and quiz paths need careful separation between an activity request and an answer submission. A click on the Quiz control should initiate question generation, not be interpreted as the learner's answer. The reviewed application contains pending-activity state and answer-handling logic to support this distinction. The reliability of all possible user phrasings and activity transitions has not been established through a full end-to-end test suite.

3.4 Input validation and persistent records

TutorInput includes defaults such as student_id='default_student', language='English', and subject='Computer Science', along with a force_new_exercise flag. The input schema tests check valid values, defaults, empty messages, messages longer than 4,000 characters, and empty student identifiers. ConversationStore persists messages, tool executions, pending quizzes, and pending exercises in SQLite. The database is local application data and should not be committed to a public repository.

3.5 Repository organisation

The reviewed project is organised into the following principal directories and files:

- agent/: intent understanding, pedagogy, planning, reflection, and validation components.
- graph/: graph construction, state, nodes, gates, and routing.
- knowledge/: JSON knowledge loading and lexical retrieval.
- llm/: local language-model interface.
- memory/: context management, conversation memory, SQLite conversation storage, and student memory.
- schemas/: Pydantic input, output, plan, reflection, response, tool, understanding, and validation schemas.
- tools/: calculator, quiz manager, quiz validator, and tool registry.
- ui/: Streamlit application, interface components, and English/Arabic localisation.
- tests/: automated tests for input schema and student memory.
- evaluation/: task definitions and a component-level evaluation runner.
- main.py: tutor execution entry point and coordination with conversation storage.

The prompts directory existed in the reviewed project listing but contained no separate prompt files at that time; prompt content was embedded in Python components. The repository also contained a local virtual environment, Python cache files, and a SQLite database. These are local runtime artefacts rather than source deliverables and should be excluded from version control.

4. Evaluation and Reliability

4.1 Evaluation approach

The project includes a small automated evaluation harness intended to make selected component checks reproducible. The task set is stored in evaluation/tasks.json, and evaluation/run_evaluation.py executes the named checks and prints a summary. The tasks cover input-schema behaviour and student-memory behaviour. The evaluation runner does not call Ollama or an external API, so these checks can run without a model service or network connection, assuming the required Python packages are installed.

4.2 Test suite results

The user executed the test suite with python -m pytest tests/ -v. The recorded result was nine tests passed. The tests comprise five input-schema checks and four student-memory checks. The input-schema tests cover a valid request, default values, rejection of an empty message, rejection of a message exceeding 4,000 characters, and rejection of an empty student identifier. The student-memory tests cover the initial state, adding a mastered concept, adding a weak concept, and avoiding duplicate concepts.

Test area	Number of tests	Recorded result
Input schema	5	Passed
Student memory	4	Passed
Total	9	9 passed

4.3 Evaluation harness results

The user also executed python evaluation/run_evaluation.py. The recorded output reported nine total tasks, nine passed, zero failed, and a 100.00% pass rate. The tasks correspond to the same two component areas as the pytest suite. The result is reproducible evidence for the selected assertions in the recorded environment.

Evaluation measure	Recorded value
Total tasks	9
Passed	9
Failed	0
Pass rate	100.00%

4.4 Interpretation of the results

The 100.00% result must be interpreted narrowly. It means that all nine implemented evaluation tasks passed in the recorded run. It does not mean that the complete tutor is 100% accurate, that all quiz questions are valid, that the language model always gives correct explanations, or that students learn more effectively after using the application. The current tests do not measure end-to-end graph routing, response factuality, mathematical equivalence across arbitrary expressions, pedagogical quality, latency, model cost, or learner outcomes.

A stronger evaluation would require a larger, independently reviewed task set covering each subject and action type. It should include expected outputs or scoring rubrics, adversarial and ambiguous prompts, incorrect student answers, repeated attempts, pending-activity transitions, malformed model outputs, tool failures, and regression

cases. Human reviewers with subject expertise should assess correctness, clarity, age appropriateness, and whether the tutor encourages reasoning rather than simply revealing answers.

4.5 Known reliability risks

- The local model may produce incorrect, incomplete, or poorly targeted explanations and quiz questions.
- The reflection parser has a permissive fallback that may treat an unparseable reflection result as sufficient; this can allow a workflow to proceed when uncertainty should instead be surfaced.
- The output validator primarily checks response structure and validity flags; it does not independently verify factual or mathematical correctness.
- The repair node, as reviewed, adds a note and adjusts validation state rather than performing a substantive regeneration and independent correction.
- The knowledge retriever uses lexical matching, so relevant material may be missed when wording differs from stored terms.
- Some action names are permitted by the action gate but were not confirmed as implemented in the act-node path.
- The calculator restricts expression syntax, but additional resource limits would be prudent for very large exponents or computationally expensive expressions.
- The local LLM request has a timeout but no confirmed retry or backoff strategy.

5. Responsible Design

5.1 Educational safety and learner agency

The tutor is intended to support learning, not replace the learner's own work or the role of a teacher. Hints, worked explanations, practice tasks, and quizzes provide different levels of support. The design objective is to make the learner participate in the solution process. In future development, the system should make its pedagogical policy more explicit and test whether hints are appropriately staged before full solutions are shown.

5.2 Accuracy, uncertainty, and verification

Language-model output is probabilistic and may contain errors. The application should therefore avoid presenting every generated statement as authoritative. Mathematical answers should be checked by deterministic tools where possible, and generated questions should be independently validated before they are stored as pending activities. Where validation is inconclusive, the tutor should communicate uncertainty or ask for clarification rather than claiming certainty. The current validation mechanisms provide partial safeguards but do not establish correctness for all subjects and all generated content.

5.3 Privacy and data handling

The current implementation stores conversation and activity records in a local SQLite database. This reduces dependence on a remote conversation-storage service, but local storage is not automatically secure. The reviewed implementation does not establish database encryption, user authentication, role-based access control, a retention schedule, or a user-facing deletion mechanism. If the application is deployed for real students, particularly minors, it should collect only necessary information, clearly explain what is stored, protect the database and backups, define retention and deletion procedures, and obtain appropriate institutional or parental permissions where required.

5.4 Human oversight and approval

The graph contains input, action, and output validation gates. These are software control points that validate requests, restrict actions, or check outputs. They are not a human approval gate. The reviewed project does not implement a step in which a teacher or other authorised person must approve an action before it occurs. This distinction is important: the report does not claim human-in-the-loop approval functionality that is not present. For deployment in a formal educational setting, teacher review and escalation paths should be considered for high-impact or disputed content.

5.5 Tool boundaries and security

The calculator uses a restricted abstract syntax tree evaluator rather than unrestricted Python evaluation, which reduces exposure to arbitrary code execution through calculator expressions. The tool registry and action gate also constrain which named actions can be selected. Nevertheless, the action allowlist must match the actions that are actually implemented, and every tool should validate its inputs independently. The current local model endpoint should not be exposed publicly without authentication, network restrictions, and suitable operational safeguards.

5.6 Accessibility, language, and fairness

The interface includes English and Arabic language options. This is a useful accessibility feature for learners who prefer either language, but bilingual labels alone do not guarantee equivalent learning quality. Some lesson content and headings remain partly in English, and the breadth of Arabic explanations has not been systematically evaluated. Future review should assess terminology consistency, readability, right-to-left layout, and whether examples are equally clear in both languages. The system should also avoid assuming that a learner's language choice, mistakes, or pace indicates ability or background.

5.7 Risk register

Risk	Potential impact	Current mitigation / status	Further action
Incorrect model-generated explanation	Learner acquires a misconception.	Output structure checks and selected tool validation; factual correctness not fully verified.	Subject-expert review and broader answer verification.
Invalid generated quiz	Learner is assessed against a flawed question or answer.	Quiz validator and bounded generation attempts for supported topics.	Expand independent validators and regression tests.
Privacy exposure	Conversation data may be accessed by an unauthorised person.	Local SQLite storage; no confirmed encryption or access-control layer.	Authentication, encryption, retention and deletion controls.
Unbounded or incorrect action routing	Unexpected workflow behaviour or wasted resources.	Action gate and bounded graph iterations.	Align allowlist with implemented tools; test all routes.
Language inequity	Arabic users receive incomplete or less clear content.	English/Arabic localisation dictionaries and some bilingual lessons.	Full bilingual content review and usability testing.

Over-reliance on tutor	Learner may accept output without checking or reasoning.	Hints and practice interactions are available.	Explicit uncertainty cues and teacher guidance.

5.8 AI-use declaration

AI tools were used as development assistance during project implementation and documentation. The project itself uses a locally hosted Qwen2.5 3B language model through Ollama for selected natural-language understanding and generation tasks. ChatGPT was also used as an assistant during development to help reason about architecture, debug code, draft and revise supporting files, and prepare documentation. The developer remains responsible for reviewing, understanding, testing, and verifying the submitted code and claims. The recorded pytest and evaluation results were produced by executing the project's local test commands. This declaration should be amended if any additional AI tools, external code sources, datasets, or generated assets were used.

6. Limitations and Next Steps

6.1 Current limitations

The project demonstrates an integrated prototype, but several limitations affect its readiness for wider use. First, the automated evaluation is component-level and small. Second, the local language model's output quality has not been measured through a blinded expert review. Third, the learner model is basic: student-memory counters are present but are not fully updated or converted into a validated mastery estimate. Fourth, the knowledge base uses lexical retrieval and has limited coverage. Fifth, the application does not include a human approval gate. Sixth, security and privacy controls suitable for a public student-facing deployment have not been demonstrated.

The project also has deployment constraints. The current configuration expects Ollama and the Qwen2.5 3B model to be available locally. A provider-switching configuration for a second cloud LLM provider was not confirmed in the reviewed code. The application has not been verified through a clean clone on a separate machine. The public GitHub repository upload and the unlisted YouTube demonstration remain submission tasks, and the report cover therefore contains placeholders for the final URLs and student metadata.

6.2 Recommended next steps

1. Complete a clean-machine installation test from a fresh repository clone, following only the README instructions.
2. Expand the evaluation task set to cover all graph routes, tool actions, quiz and exercise flows, error recovery, and multilingual prompts.
3. Introduce expert-reviewed answer rubrics and measure factual correctness, mathematical correctness, pedagogical usefulness, and response clarity.
4. Improve student-memory updates so attempts and correct attempts are recorded consistently and mastery changes are evidence-based.
5. Replace or strengthen lexical retrieval with structured metadata and, if justified by the project scope, semantic retrieval.
6. Implement a genuine human review or approval workflow if the course requirements or deployment context require one.
7. Add authentication, database protection, retention limits, deletion controls, and a privacy notice before deployment with real student data.

8. Align the action allowlist with fully implemented tools and make unsupported actions fail safely.
9. Add resource limits and stronger error handling for computational tools and local model requests.
10. Complete bilingual content review and usability testing with representative learners.
11. Upload the final source repository, record the required ten-minute demonstration, and insert the verified links into the cover page.

7. Conclusion

AI Educational Tutor demonstrates a structured approach to an agentic educational assistant. It combines a Streamlit learning interface, a LangGraph workflow, a local language model, knowledge retrieval, selected tools, SQLite persistence, and Pydantic schemas. Its main contribution is the integration of these elements into a bounded tutoring workflow that distinguishes learner intents and supports explanations, hints, practice, and quizzes. The nine passing tests and nine passing evaluation tasks provide evidence that selected input-validation and student-memory behaviours work as expected in the recorded environment.

The evidence must nevertheless be interpreted within its scope. The project has not yet demonstrated comprehensive end-to-end reliability, validated educational outcomes, a human approval gate, or deployment readiness for real student data. These are explicit limitations rather than capabilities to be assumed. With broader evaluation, stronger learner modelling, privacy safeguards, expert review, and deployment verification, the prototype could be developed into a more reliable educational support system.

References

1. LangGraph documentation. LangChain. https://langchain-ai.github.io/langgraph/ (accessed during project development; replace or supplement with the exact versioned documentation consulted).

2. Streamlit documentation. https://docs.streamlit.io/ (accessed during project development; add specific pages consulted if required by the module referencing style).

3. Ollama documentation. https://ollama.com/ (accessed during project development; add the exact model and API documentation pages consulted).

4. Pydantic documentation. https://docs.pydantic.dev/ (accessed during project development).

5. Python documentation. https://docs.python.org/3/ (accessed during project development).

6. pytest documentation. https://docs.pytest.org/ (accessed during project development).

Note: Confirm the module's required referencing style and add any external sources, datasets, code, or design materials actually used. Do not cite sources that were not consulted.

Appendix A — Risk Register, Guardrails, and Human Oversight

A.1 Risk summary

The principal risks are incorrect educational content, invalid generated assessments, privacy exposure, misrouted actions, incomplete bilingual support, and learner over-reliance. The current prototype addresses some risks through structured input schemas, an action allowlist, bounded execution, a restricted calculator, quiz validation for selected topics, and output checks. These safeguards reduce specific classes of failure but do not eliminate the risks.

A.2 Implemented guardrails

- Input validation checks required identifiers, message presence, message length, and subject values.
- Action gating restricts the workflow to a defined list of action names.
- Graph execution is bounded by an iteration limit and a recursion limit.
- The calculator accepts a limited arithmetic syntax through AST inspection rather than arbitrary eval execution.
- Quiz generation uses bounded attempts and validation for supported question types.
- Output gating checks for a non-empty response and validation status before returning it.
- Conversation and pending-activity state are stored locally in SQLite.

A.3 Human oversight status

No human approval gate is implemented in the current project. Input, action, and output gates are automated software checks and must not be described as teacher approval. A future human oversight design could allow a teacher to review flagged explanations, disputed answers, or assessment items before they are used in a formal learning activity. Such a feature would need explicit roles, review records, and a clear policy for when approval is mandatory.

A.4 Deployment safeguards recommended

- Do not use identifiable student records in the prototype without an approved privacy and security arrangement.
- Keep the Ollama service bound to a trusted local environment unless secure remote access has been configured.
- Do not commit .env files, local databases, virtual environments, or credentials to the public repository.
- Provide a clear notice that model-generated content may be wrong and should be checked.
- Provide a route for students to report incorrect or inappropriate content to a responsible educator.
- Define data retention, deletion, backup, and access policies before collecting real learner data.

Appendix B — AI Use and Verification Declaration

AI assistance used: ChatGPT was used as a development and documentation assistant for architecture discussion, debugging support, preparation of supporting repository files, and report drafting. The application integrates Qwen2.5 3B locally through Ollama for language-model tasks. If other AI assistants or services were used, list them here before submission.

Purpose: AI assistance supported implementation planning, troubleshooting, explanation of technical concepts, and preparation of documentation. The local model provides selected natural-language interpretation and generation within the tutor application.

Verification: The developer reviewed and ran the project's component tests and evaluation harness. The recorded commands were python -m pytest tests/ -v and python evaluation/run_evaluation.py. Both produced nine passing checks/tasks. These checks verify the tested input-schema and student-memory behaviours only. They do not verify all generated tutoring content or the full end-to-end application. The developer must review the complete source code, confirm all report statements, and ensure that all external sources are acknowledged.

Student confirmation: I confirm that I have reviewed this report and the submitted implementation, understand the work being submitted, and have identified the AI assistance and external sources used.

Student name: _____

Student number: _____

Date: _____

Submission completion notes (remove this page before final submission)

This working report is editable. Before submission, complete the cover-page placeholders: student name, student number, team ID, public GitHub repository URL, unlisted YouTube video URL, and final word count. Check that the repository and video links open for the marker. The course brief asks for one PDF report of approximately 3,000–4,000 words plus the responsible-design appendix; review the final word count in Word and adjust the report to the module's counting convention. Remove this completion-notes page before exporting the final PDF.

Important factual boundary: the recorded 100.00% evaluation is for nine component-level tasks concerning input validation and student-memory behaviour. It is not an end-to-end accuracy score or a measure of learning outcomes. The project currently has no human approval gate, and the clean-machine clone test, public repository upload, and video recording were not confirmed complete when this report was drafted.